

Lab-PM: Bygg ett terminalbaserat chattforum via SSH

1. Syfte

Syftet med labben är att själv konstruera ett enkelt terminalbaserat chattforum där flera användare kan ansluta till samma chattrum via en Linux-server.

Tanken är att du inte bara ska köra färdig kod, utan själv behöva resonera kring:

- hur en server tar emot flera simultana anslutningar
- hur meddelanden kan skickas mellan klienter
- hur terminalprogram kan kommunicera över nätverk
- hur SSH kan användas som åtkomstväg
- hur programmet kan köras stabilt på en server

Labben är därför formulerad som en konstruktionsuppgift. Du får mål, krav, struktur och testfall, men du ska själv skriva implementationen.

2. Målbild

När labben är klar ska systemet fungera ungefär så här:

```
Användare 1 → SSH till server → startar chatten → skriver meddelanden  
Användare 2 → SSH till server → startar chatten → ser och skriver meddelanden  
Användare 3 → SSH till server → startar chatten → deltar i samma rum
```

Alla användare ska hamna i samma chattrum.

Ett rimligt användarflöde är:

```
ssh användare@server-ip  
chat
```

Därefter ska användaren kunna välja ett namn och börja skriva meddelanden.

3. Avgränsning

Den här labben ska inte bygga ett fullständigt Discord, IRC eller Matrix-alternativ. Målet är en enkel prototyp.

Systemet behöver inte ha:

- grafiskt gränssnitt
- användarkonton i själva chatten
- lösenordsskydd i chatten
- flera rum
- databas
- kryptering utöver SSH
- modereringsfunktioner
- historik

Systemet ska däremot ha:

- en serverprocess
- flera samtidiga klienter
- textmeddelanden som skickas till alla anslutna användare
- möjlighet att lämna chatten utan att servern kraschar
- enkel felsökbarhet via terminalen

4. Rekommenderad teknisk lösning

En rimlig lösning är att bygga två program:

1. Chattserver

2. Körs på Linux-servern.

3. Lyssnar på en lokal TCP-port.

4. Tar emot flera klienter.

5. Skickar inkommande meddelanden vidare till övriga klienter.

6. Chattklient

7. Körs efter SSH-inloggning.

8. Ansluter till chattservern.

9. Läser in text från tangentbordet.

10. Visar inkommande meddelanden.

Rekommenderat språk: **Python**.

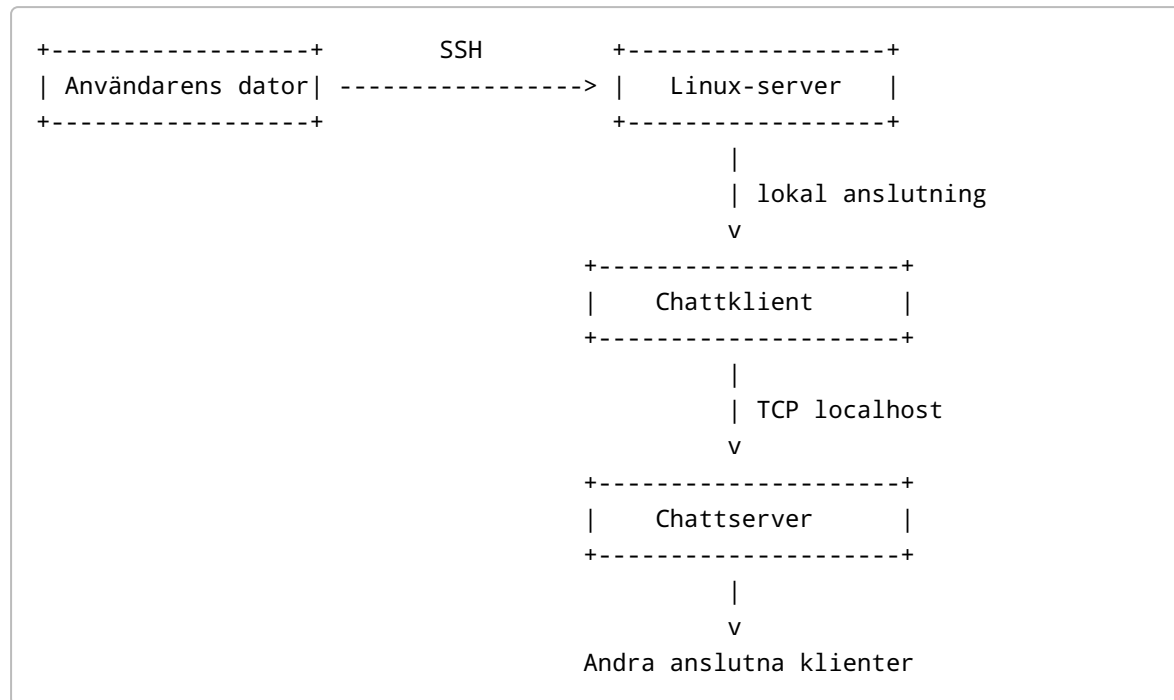
Motivering:

- Python finns ofta redan på Linux-system.
- Standardbiblioteket räcker långt.
- Det är lätt att felsöka.
- Det går snabbt att testa små delar.

Du får använda andra språk om du vill, till exempel Go, Rust, JavaScript/Node eller C. Labben är dock skriven med Python i åtanke.

5. Systemskiss

Översikt:



Viktig idé:

Chattservern behöver inte exponeras direkt mot internet. Den kan lyssna på `127.0.0.1`, eftersom användarna först loggar in via SSH och därefter kör klienten lokalt på servern.

6. Förkunskaper

Du bör ha grundläggande koll på:

- Linux-terminalen
- SSH
- filer och mappar
- enklare programmering
- loopar, funktioner och felhantering
- vad en TCP-port är på en ungefärlig nivå

Du behöver inte kunna avancerad nätverksprogrammering innan labben. Poängen är att undersöka det under arbetet.

7. Förbered servern

Logga in på din server:

```
ssh användare@server-ip
```

Skapa en projektmapp, till exempel:

```
mkdir -p ~/projects/terminal-chat  
cd ~/projects/terminal-chat
```

Kontrollera att Python finns:

```
python3 --version
```

Skapa gärna en virtuell miljö:

```
python3 -m venv .venv  
source .venv/bin/activate
```

Du får själv välja filstruktur, men en rimlig start är:

```
terminal-chat/  
├─ server.py  
├─ client.py  
├─ README.md  
└─ notes.md
```

8. Del 1: Designa protokollet

Innan du skriver kod ska du bestämma hur klient och server ska prata med varandra.

Besvara följande frågor i `notes.md`:

1. Vad ska klienten skicka när den ansluter?
2. Hur väljer användaren namn?
3. Hur ska ett vanligt meddelande se ut?
4. Hur ska en användare lämna chatten?
5. Ska servern skicka systemmeddelanden, till exempel "Mats joined"?
6. Ska avsändaren själv få tillbaka sitt eget meddelande?
7. Hur ska tomma meddelanden hanteras?

Exempel på ett enkelt textbaserat protokoll:

```
Först skickar klienten sitt användarnamn.  
Därefter skickas varje rad användaren skriver som ett meddelande.  
Om användaren skriver /quit stängs anslutningen.
```

Detta är inte färdig kod, utan en specifikation av hur programmen ska bete sig.

9. Del 2: Bygg en minimal server

Börja med en server som bara klarar en klient.

Servern ska kunna:

- starta utan fel
- lyssna på en port
- acceptera en klientanslutning
- ta emot text från klienten
- skriva ut texten i serverns terminal
- stänga anslutningen när klienten kopplar från

Testa detta innan du går vidare.

Kontrollfrågor

- Vilken IP-adress lyssnar servern på?
- Vilken port använder du?
- Vad händer om porten redan används?
- Vad händer om klienten stänger anslutningen oväntat?

Förväntad testbild

Terminal 1:

```
Server started  
Client connected  
Received: hej  
Received: test  
Client disconnected
```

Terminal 2:

```
Connected to server  
> hej  
> test  
> /quit
```

10. Del 3: Bygg en minimal klient

Klienten ska kunna:

- ansluta till servern
- läsa text från användaren
- skicka text till servern
- avsluta när användaren skriver `/quit`
- hantera att servern inte är igång

Testa klienten mot servern från del 2.

Kontrollfrågor

- Vad händer om klienten startas när servern inte kör?
- Vad händer om servern stängs av medan klienten är ansluten?
- Behöver klienten visa ett tydligt felmeddelande?

11. Del 4: Stöd flera klienter

Nu ska servern kunna hantera flera samtidiga klienter.

Detta är labbens viktigaste steg.

Du behöver lösa två problem:

1. Servern måste kunna ta emot flera anslutningar utan att fastna på den första.
2. När en klient skickar ett meddelande ska servern kunna skicka det vidare till andra klienter.

Det finns flera möjliga tekniska vägar:

- trådar
- `asyncio`
- processer
- select/poll

För denna labb är trådar eller `asyncio` mest rimligt.

Krav

När klient A skriver ett meddelande ska klient B kunna se det.

Exempel:

Klient A:

Mats: hej

Klient B ska då se:

```
Mats: hej
```

Kontrollfrågor

- Hur håller servern reda på alla anslutna klienter?
- Vad händer om en klient kopplas bort?
- Ska servern skicka meddelandet till alla, eller alla utom avsändaren?
- Vad händer om två användare skriver nästan samtidigt?

12. Del 5: Hantera användarnamn

Lägg till stöd för användarnamn.

När en klient ansluter ska den få välja ett namn.

Servern ska sedan använda namnet när meddelanden skickas vidare.

Exempel:

```
[12:41] mats: hej  
[12:42] anna: tjena
```

Krav

- Tomt användarnamn ska inte krascha programmet.
- Två användare med samma namn ska inte krascha programmet.
- Servern ska kunna visa när någon ansluter.
- Servern ska kunna visa när någon lämnar chatten.

Designval

Du får själv välja om dubbla namn ska tillåtas eller inte.

Om de tillåts kan det bli otydligt.

Om de inte tillåts behöver servern säga till klienten att välja ett annat namn.

13. Del 6: Gör chatten användbar i terminalen

När klienten både ska läsa tangentbordsinput och ta emot meddelanden från servern uppstår ett praktiskt problem:

Användaren kan skriva samtidigt som nya meddelanden kommer in.

Du behöver därför tänka igenom hur klienten ska vara uppbyggd.

Minimikrav:

- klienten ska kunna visa inkommande meddelanden
- klienten ska kunna skicka användarens egna meddelanden
- klienten ska inte låsa sig medan den väntar på input

Möjliga lösningar:

- en tråd som läser från servern och en annan del som läser från tangentbordet
- `asyncio`
- ett enklare terminalbibliotek

Första versionen behöver inte vara perfekt visuellt. Det räcker att den fungerar.

14. Del 7: Skapa ett `chat`-kommando

När klienten fungerar kan du skapa ett enkelt kommando så att användaren slipper gå in i projektmappen manuellt.

Målet är att detta ska fungera efter SSH-inloggning:

`chat`

Du behöver själv undersöka:

- var egna kommandon brukar placeras på Linux
- hur man gör en fil körbar
- hur ett shell-script kan starta klientprogrammet
- hur scriptet hittar rätt projektmap

Kontrollfrågor

- Vad gör `$PATH`?
- Varför fungerar kommandon i `/usr/local/bin` ofta direkt?
- Vilka rättigheter behöver scriptet ha?

15. Del 8: Kör servern stabilt

Under utveckling kan du starta servern manuellt.

För mer permanent drift bör servern startas automatiskt och startas om om den kraschar.

Ett rimligt sätt på Ubuntu är att använda `systemd`.

Du ska skapa en tjänst som:

- startar chattservern
- körs som en vanlig användare, inte root
- startar om servern vid krasch
- kan aktiveras vid boot

Saker att undersöka

- Vad är en `systemd`-service?
- Vad betyder `ExecStart`?
- Vad betyder `WorkingDirectory`?
- Vad betyder `Restart=always`?
- Varför bör tjänsten inte köras som root?

Kommandon du sannolikt behöver använda

```
systemctl status namn.service
sudo systemctl start namn.service
sudo systemctl stop namn.service
sudo systemctl restart namn.service
sudo systemctl enable namn.service
journalctl -u namn.service -f
```

Du ska själv skriva servicefilen.

16. Del 9: SSH som åtkomstväg

Den enklaste lösningen är att användaren först loggar in via SSH och sedan själv skriver:

```
chat
```

Detta räcker för första versionen.

En mer avancerad lösning är att skapa en särskild Linuxanvändare som automatiskt hamnar i chatten vid SSH-inloggning.

Exempel på målbild:

```
ssh chatuser@server-ip
```

Då ska användaren direkt hamna i chatten, utan att få ett vanligt shell.

Detta kan göras med SSH-inställningar, till exempel `ForceCommand`.

Viktig säkerhetsregel

Testa aldrig SSH-konfiguration utan att ha en extra fungerande SSH-session öppen. Annars riskerar du att låsa ute dig själv.

Frågor att undersöka

- Vad gör `ForceCommand`?
- Vad är skillnaden mellan att ge användaren ett shell och att tvinga fram ett kommando?
- Vilka risker finns med att låta externa personer logga in på servern?
- Hur kan SSH-nycklar användas istället för lösenord?

17. Del 10: Säkerhet och nätverk

Eftersom chatten nås via SSH behöver chattservern inte vara öppen direkt mot internet.

Rekommenderad design:

SSH: öppen mot nätverket
Chattserver: endast localhost

Det betyder:

Chattservern lyssnar på 127.0.0.1
Inte på 0.0.0.0

Kontrollfrågor

1. Vad är skillnaden mellan `127.0.0.1` och `0.0.0.0`?
2. Varför är det säkrare att bara lyssna lokalt?
3. Vilken port måste vara öppen i brandväggen?
4. Behöver chattporten öppnas i UFW?
5. Vad händer om någon försöker ansluta direkt till chattporten utifrån?

18. Testplan

Du ska testa systemet stegvis.

Test 1: Servern startar

Förväntat resultat:

- servern startar utan fel
- den skriver ut att den lyssnar på vald port

Test 2: En klient kan ansluta

Förväntat resultat:

- klienten ansluter
 - servern visar att en klient anslutit
 - klienten kan skicka ett meddelande
-

Test 3: Två klienter kan kommunicera

Förväntat resultat:

- klient A skriver ett meddelande
 - klient B ser meddelandet
 - klient B svarar
 - klient A ser svaret
-

Test 4: En klient lämnar

Förväntat resultat:

- klienten kan skriva `/quit`
 - servern kraschar inte
 - övriga klienter är fortfarande anslutna
-

Test 5: Servern startas via systemd

Förväntat resultat:

- servern kan startas med `systemctl`
 - status visar att tjänsten kör
 - loggar syns via `journalctl`
-

Test 6: Anslutning via SSH

Förväntat resultat:

- användare kan logga in via SSH
 - användare kan starta chatten med `chat`
 - flera SSH-sessioner kan delta samtidigt
-

19. Felsökningsstrategi

När något inte fungerar, felsök i denna ordning:

1. Kör servern manuellt i terminalen.
2. Starta en klient lokalt på samma server.
3. Testa med två klienter lokalt.
4. Testa via SSH.
5. Testa `chat`-kommandot.
6. Testa systemd-tjänsten.
7. Läs loggar med `journalctl`.

Undvik att felsöka allt samtidigt. Isolera ett problem i taget.

20. Vanliga problem att själv kunna lösa

Servern startar inte

Möjliga orsaker:

- porten används redan
 - fel Python-version
 - syntaxfel
 - fel sökväg
 - saknade rättigheter
-

Klienten kan inte ansluta

Möjliga orsaker:

- servern kör inte
 - fel port
 - fel host
 - servern lyssnar inte där du tror
-

Bara en klient fungerar

Möjliga orsaker:

- servern hanterar klienter sekventiellt
 - servern fastnar i en loop för första klienten
 - ingen trådning eller asynkron hantering används
-

Meddelanden syns inte hos andra

Möjliga orsaker:

- servern sparar inte listan över anslutna klienter
 - broadcast-funktionen skickar inte till rätt anslutningar
 - data flushas inte korrekt
 - klienten lyssnar inte på inkommande data samtidigt som den väntar på input
-

Systemd-tjänsten fungerar inte

Möjliga orsaker:

- fel `ExecStart`
 - fel `WorkingDirectory`
 - tjänsten körs som fel användare
 - relativ sökväg används där absolut sökväg krävs
 - Pythonmiljön hittas inte
-

21. Dokumentation som ska lämnas in

Skriv en kort README eller labbrapport med följande delar:

1. **Syfte**
2. Vad skulle systemet göra?
3. **Systemdesign**
4. Hur är server och klient uppdelade?
5. Hur går kommunikationen till?
6. **Protokoll**
7. Hur ser meddelanden ut?
8. Hur ansluter och lämnar användare?
9. **Implementation**
10. Vilka tekniska val gjorde du?
11. Använde du trådar, `asyncio` eller något annat?
12. **Testning**

13. Vilka tester gjorde du?
 14. Vad fungerade?
 15. Vad fungerade inte först?
 16. **Säkerhet**
 17. Varför används SSH?
 18. Varför bör chattservern bara lyssna lokalt?
 19. **Vidareutveckling**
 20. Vad skulle du förbättra om du fortsatte?
-

22. Bedömningskriterier

En enkel godkänd lösning bör uppfylla:

- servern kan startas
- minst två klienter kan ansluta samtidigt
- meddelanden skickas mellan klienterna
- klienter kan lämna utan att servern kraschar
- programmet kan användas via SSH
- lösningen är dokumenterad

En starkare lösning kan dessutom ha:

- tydliga användarnamn
 - tidsstämplar
 - kommandon som `/quit` och `/who`
 - bättre felhantering
 - systemd-tjänst
 - särskild SSH-användare som hamnar direkt i chatten
 - sparad historik
 - tydlig README
-

23. Reflektionsfrågor

Besvara gärna dessa efter labben:

1. Vad var svårast: serverdelen, klientdelen eller SSH-delen?
2. Varför räcker det inte att servern bara hanterar en klient i taget?
3. Vad händer om en klient kraschar mitt under en sändning?
4. Hur skulle du lägga till flera chattrum?
5. Hur skulle du spara meddelanden i SQLite?
6. Hur skulle du hindra användare från att skriva extremt långa meddelanden?

7. Hur skulle du moderera chatten?
 8. Hur skiljer sig denna lösning från ett webbforum?
 9. Vilka delar skulle behöva ändras om chatten skulle vara publik?
 10. Vad gör SSH i denna arkitektur?
-

24. Möjliga extrauppgifter

Extra 1: `/who`

Lägg till ett kommando som visar vilka användare som är anslutna.

Extra 2: Chathistorik

Spara de senaste 20 meddelandena i minnet och visa dem för nya användare när de ansluter.

Extra 3: Loggfil

Spara alla meddelanden i en loggfil på servern.

Fundera på:

- var filen ska ligga
 - vilka rättigheter den ska ha
 - om loggen ska kunna växa obegränsat
-

Extra 4: SQLite

Spara meddelanden i en SQLite-databas.

Fundera på tabeller som:

```
users
messages
rooms
```

Du behöver inte bygga full användarhantering, men du kan börja modellera datan.

Extra 5: Flera rum

Lägg till stöd för flera chattrum.

Exempel:

```
/join general  
/join linux  
/join random
```

Extra 6: Direkt SSH-chatt

Skapa en separat SSH-användare som automatiskt startar chatten vid inloggning.

Mål:

```
ssh chatuser@server-ip
```

ska öppna chatten direkt.

25. Slutsats

I denna labb ska du bygga ett enkelt terminalbaserat chattforum som nås via SSH. Fokus ligger på att förstå arkitekturen bakom en chatt: en server som håller koll på anslutna klienter, ett enkelt textprotokoll och klienter som både kan skicka och ta emot meddelanden.

Labben är avsiktligt öppen. Du ska själv fatta flera designbeslut och lösa implementationen steg för steg. Det gör uppgiften mer lik ett riktigt mindre programmeringsprojekt än en ren kopiera-och-kör-labb.